

Kidde Customer Solutioning — Prep Doc

Holden Ottolini

Anthropic SA Final Round · April 2026

Contents

0. The prompt (verbatim)	2
0.5 Recruiter feedback — verification checklist	2
1. The three-slide spoken arc (10–15 minutes)	3
Slide 1 — The Challenge (~4 minutes)	3
Slide 2 — The Technical Solution (~5–6 minutes) [where technical depth lands]	3
Slide 3 — The Outcome (~3 minutes)	6
2. Stakeholder map (keep this in your head)	6
3. Anticipated probes — rehearsed answers	7
a. “Why Yext specifically, not a headless CMS + custom React?”	7
b. “Why Yext’s native AI search, not Elasticsearch, Algolia, or a vector store?”	7
c. “Walk me through the rescope decision in more detail.”	8
d. “How did you develop the champion? — business side AND technical side”	8
e. “What would you do differently?”	9
f. “Where did this almost fall apart?”	9
g. “How did you measure ROI for the customer?”	9
h. “Strategically, why did this customer expand with you instead of bringing it in-house?”	9
i. “If we gave you Claude Code for this engagement, where would it have mattered most?”	10
j. “What does this engagement say about how you’d show up at Anthropic?”	10
4. Security & compliance — the parent corporate story	10
5. Strategic importance — why this deal mattered (to them and to Arc4)	11
6. Numbers you must have memorized	12
7. Anchor phrases to use on-panel	12
8. Kill list (don’t say)	12
9. Technical depth territories (where to dig if they push beyond CMS + API)	12
10. The “I don’t know” discipline	13
11. T-minus checklist	13

0. The prompt (verbatim)

Be prepared to speak on a complex customer or partner engagement where you worked to solve a technical problem. Be prepared to walk through the background, the engagement strategy, the technical solution (at an architecture level), the outcome and impact. Please come prepared with a diagram or other visual content to help tell the story (preferably from existing work and anonymized). We recommend coming prepared with ~3 slides covering the customer challenge, technical solution, and business outcome.

Target length: 10-15 minutes of story, 15-20 minutes of Q&A. **What they're evaluating:** champion development, technical depth, tradeoffs, security/compliance, strategic importance, ROI. You were told directly: *push beyond CMS + API integrations*. So the spoken version has to surface the deeper engineering layer — schema governance, multi-tenant architecture, deployment discipline, observability — not just platform choice.

Anonymization: deck uses “Global Consumer Safety Brand, subsidiary of a Fortune 500 building-systems parent.” Live delivery: you can use Kidde/Carrier names if the panel already knows, or keep it anonymized. Default to anonymized unless asked.

0.5 Recruiter feedback — verification checklist

Recruiter bullet	Where it's addressed
Architectural diagram + 10-15 min walk	S2 of deck (AI search architecture), §1 of this doc
Customer story + initial pain points	S1 of deck, §1 Challenge walk
How it was applied (the solution)	S2 of deck, §1 Solution walk
ROI	S3 of deck, §1 Outcome walk, §3g, §6
Champion development (business + technical)	§3d (two-track: commercial champion + technical champion)
Technical tradeoffs	§1 Solution walk (three decisions), §3a, §3b, §3c
Security & compliance	§4 (dedicated section)
Strategic importance of the deal	§5 (dedicated section), §3h
Technical depth as evaluation axis	S2 AI search architecture + §2 probes + §4 security + §7 technical-depth territories

If a panelist asks something not in this grid, say so, then commit to following up — see §8.

1. The three-slide spoken arc (10-15 minutes)

Deck is exactly three slides. Per the prompt. The slides are dense on purpose; the story lives in the walk, not on the page.

Slide 1 — The Challenge (~4 minutes)

What's on the slide: headline narrative, five stakeholder groups, four underlying pain points.

How to walk it:

"This is a case study from my own company, Arc4. The engagement I learned the most from, and the one I think is the closest analog to how I'd work with a customer at Anthropic.

Late 2023: a subsidiary of a Fortune 500 building-systems manufacturer came to us for a single consumer-facing website rebuild. \$140K scope, one property, one stakeholder on the kickoff call.

By month three it was obvious that wasn't the real engagement. The real engagement was five stakeholder groups with different incentives — the consumer BU who signed the SOW and wanted speed; the Global Solutions BU on the B2B side with a different data model; an adjacent portfolio brand that wanted on the same platform; the agency of record who owned the retainer and was our paying counterparty; and parent corporate — brand, security, legal, procurement.

Underneath that, four things were actually broken. Ten properties on four CMS platforms, inconsistent schemas, no source of truth. A marketing ops team that ran content day-to-day with no appetite for Git or React, which meant any solution requiring dev help for content changes was dead on arrival. Cross-locale SEO debt — broken hreflang, missing structured data, six countries. And no deployment discipline — local-machine deploys, no env parity, no rollback, no audit trail. Parent corporate was never going to approve expansion on that foundation.

The first problem wasn't technical. It was figuring out who had to say yes, and for which decision. That map drove every architectural decision that followed."

Slide 2 — The Technical Solution (~5-6 minutes) [where technical depth lands]

Framing: this slide tells the story of **AI search over an entity graph**. The CMS was Yext, but the real engineering was the data modeling — taxonomy design, typed entities, schema registry, IDR — that made the AI search actually trustworthy. Lead with the data, not the platform.

What's on the slide: a three-tier pipeline (Sources → Entity Layer → Yext AI Search Platform), the middle tier highlighted because that's where the work hap-

pened. Right-side dark panel has three search-specific decisions. Bottom bar has implementation disciplines.

How to walk it:

“The deliverable the panel will hear people talk about is the sites. The thing that actually made those sites work at scale — and made everything else after them possible — was AI search over a clean entity graph.

Start at the bottom of the diagram and read up.

Sources were a mess. PIM for SKUs and specs. DAM for hero imagery and safety assets. A support corpus — KBs, manuals, safety docs — that lived in a Zendesk-adjacent system. And legacy CMS dumps from ten sites on four platforms, none of them agreeing on what a ‘product’ was.

The middle tier is where the work happened, and it’s the coral-highlighted row on the slide on purpose. Four moves.

One — taxonomy design. We defined the entity types by intent, not by database convenience: Product, Category, Article, SKU, Locale, Safety Guide. Each one typed, each one with relationships to the others. This was the fight — everybody had a different mental model of what a ‘product family’ was, and we had to pick one and commit.

Two — typed entities. Fields, relationships, foreign-key constraints. Not just ‘a product has a name’ — a product belongs to a category, has variants, has related articles, has locale-specific copy, ships from specific SKUs. Relationships are what makes the AI search reason, not keyword-match.

Three — a schema registry. Versioned, reviewed, diffed. Changing a type was a PR that had to be approved. This is what made ten properties stop drifting away from each other.

Four — identity resolution and dedupe. The same physical part was a different SKU in three regions with three slightly different names. Without IDR, search returns four results for one thing; with it, one canonical product entity with regional variants underneath.

Once that middle tier existed, the top tier — Yext itself — mostly configured itself. **Entity graph, multi-tenant, partitioned by property. Search terms and synonyms** — ‘smoke detector’ maps to ‘alarm’, ‘CO’ maps to ‘carbon monoxide’, regional language variants for each locale. **Definitions and intents** — what a query *means* per entity type, so ‘battery replacement’ routes to Support Article, not Product. **Structured data out** — same graph feeds on-site search, hreflang-aware SEO, JSON-LD for SERP, and an LLM-ready JSON surface we stood up in 2024 for the parent’s internal assistants.

Three decisions carried the search — and this is where I want to spend most of the technical time.

One: entity taxonomy over a flat catalog. Flat index means you write custom relevance code per property, because every property’s definition of ‘related’ is different. Typed nodes let intent route itself. A query for ‘kitchen

smoke alarm installation’ is three intents — product filter, location filter, support article — and the taxonomy sends each one to the right entity type. The trade: upfront modeling cost. Weeks of workshops with marketing, support, and legal to agree on what a ‘category’ was. Worth it.

Two: search terms as config, not code. Synonyms, definitions, intent mappings all live in Yext configuration, not in application code. That means marketing ops tunes relevance — which terms are synonyms for which SKUs, what a regional customer actually calls a product — without a dev cycle. Engineering doesn’t bottleneck the relevance loop. This is the decision that made the system sustainable at ten properties with a five-person ops team. Tradeoff: we gave up the option to build custom learned ranking. Given the consumer-product-lookup relevance bar, that was the right trade.

Three: one entity graph, many surfaces. The same graph feeds on-site search, SEO structured data, voice-assistant payloads, and LLM-ready JSON. One source of truth instead of four. Every time a taxonomy change shipped, every surface got it for free. The discipline was brutal — no surface was allowed to maintain its own side-schema, ever. But it’s what made the 2024 LLM-ready payload a three-week project instead of a six-month rebuild.

Across the bottom of the slide is the delivery discipline that made the search trustworthy — schema as contract, IDR governance, a synonym-and-definition review cadence, a per-locale relevance eval set, and hreflang plus structured data validation in CI. That’s the layer beyond CMS and API integrations that made the search work the same way in six languages without a human re-tuning it every week.”

If the panel probes deeper on the architecture, these are the probes to be ready for:

- **Relevance evaluation:** per-locale eval set of ~200 query/expected-entity pairs per property, run on every schema or synonym change. P@1 and MRR tracked per locale. Regressions block the PR.
- **Identity resolution in practice:** deterministic on part number + region + variant hash; fuzzy fallback with a human review queue for dupes the rules can’t resolve.
- **Data classification:** public product data / PII from inquiries / commercial-sensitive B2B data separated at the schema level, not just the UI.
- **Tenancy:** per-property partitioning with shared schema; vendor portal on its own auth domain; no cross-BU leakage at the query layer.
- **Schema evolution:** additive by default; breaking changes require a migration script that’s rehearsed in staging first and rolled forward-compatible for one release before removal.
- **Why not a vector store:** we considered embedding product descriptions for semantic search circa 2024. Held off because the typed taxonomy plus curated synonyms hit the relevance bar with orders-of-magnitude less operational cost and full explainability for the legal team. The revisit trigger would be natural-language product questions that the synonym layer can’t absorb.

- **Zero data incidents across 18 months; two near-misses (schema drift on Canada clone, DNS cutover on UK) caught by the deployment checklist.**

Don't cover these unprompted — they eat the time. Have them ready if asked.

Slide 3 — The Outcome (~3 minutes)

What's on the slide: four hero stats (15x, \$2.4M, 17, 0), three outcome bands (reach, commercial, methodology), the three patterns.

How to walk it:

"\$140K became \$2.4M. 15x account growth in 18 months. 17 distinct deals. Zero churn.

On reach: ten properties live across six countries, consumer, B2B, and adjacent brand on one platform, one schema, one deployment pipeline.

Commercially: project work converted to retainers. SEO and managed services became ARR. Dedicated data pod added in year two.

And the piece I care most about — the delivery system became Arc4's company standard. Schema as contract, IDR log, deployment checklist. Every new enterprise engagement starts from it. The next customer benefits from this customer's hardest month.

Three patterns held across every expansion. Treat each expansion as a new problem, not a renewal — that's what kept zero churn real. Scope hard even if it costs revenue — rescoping the global merge was a smaller deal that quarter and a larger architecture over the year. And institutionalize the lessons.

Those same three patterns are how I'd show up for a customer at Anthropic."

2. Stakeholder map (keep this in your head)

Group	What they wanted	How we managed them
Consumer BU (US)	Speed, clean rebuild	Gave them launch on time by rescoping the global merge out
Global Solutions BU	Their own data model on same platform	Phase 2 with its own schema contract, not merged
Adjacent Brand BU	On the same platform, own GTM	Per-property partition, shared schema
Agency of Record	Predictable retainer, no surprises	Clipboard system, weekly cadence

Group	What they wanted	How we managed them
Parent Corporate	Security, brand, procurement gate	Deployment audit trail, SIEM feed, annual review

Champion: the VP at the Consumer BU who signed the original SOW. She trusted us because we rescoped against our own revenue in quarter two. Every expansion after that was her opening the door.

3. Anticipated probes — rehearsed answers

a. “Why Yext specifically, not a headless CMS + custom React?”

“Three reasons. One, the authoring team was non-technical — headless stacks require dev support for every content change, which caps growth at the engineering team’s size. Two, native SEO schema output — we got structured data correct on day one, not as a retrofit. Three, PIM and DAM connectors existed, which meant we didn’t rebuild integrations that already worked. The trade was less UI flexibility. For this customer — ten properties, small ops team, SEO as primary growth channel — that trade was right. For a media company that lives on custom UI, I’d make a different call.”

b. “Why Yext’s native AI search, not Elasticsearch, Algolia, or a vector store?”

“Three layers to the answer.

Against Elastic or Algolia: running a separate search system meant a second schema, a second sync pipeline, and a second on-call surface. The entity graph was already the source of truth for the sites; keeping search on the same graph meant one data model, one deploy target. We gave up fine-grained learned ranking to get operational simplicity. For consumer product lookup with a curated synonym layer, that was the right trade.

Against a vector-store / embedding approach: we evaluated it in 2024. A typed taxonomy plus curated synonyms and definitions hit our relevance bar — measured against a per-locale eval set — with full explainability for the legal team and orders of magnitude less operational cost. Vectors would have forced us to defend ‘why did the system return this’ to compliance in six jurisdictions. The revisit trigger would be natural-language product questions that our synonym layer can’t absorb, or personalized discovery — neither of which was the actual customer need.

Why Yext as the platform: entity-native from day one. The search terms, synonyms, and definitions are first-class configuration, not bolt-ons. And it let a non-technical ops team own relevance tuning, which was the actual scaling constraint.”

c. “Walk me through the rescope decision in more detail.”

“Global Solutions came in hot in Q1 of the consumer launch. The ask was to merge both BU data models into one global launch. I ran the math: merging meant a schema rewrite, four-month delay on consumer, and one data model that neither BU fully owned. I went to the consumer VP first, because she’d signed the SOW and launch delay hit her hardest. Then to Global Solutions, and framed it as them getting their own phase with their own data contract, not a dilution of the consumer launch. The hardest conversation was with the agency of record, because a smaller Q1 deal hit their numbers. I offered to lock in the Global Solutions phase as a committed Q2 scope in exchange. Everyone got something they could defend to their own leadership.”

d. “How did you develop the champion? — business side AND technical side”

There were two champions. Treat them as separate engineering problems.

“Commercial champion: the Consumer BU VP who signed the original SOW. Three moves.

One, I rescope against my own revenue in quarter two — pulling the global merge out cost us the bigger Q1 deal but let her launch on time. That’s the moment she started trusting me. Once a vendor puts your outcome ahead of their quarter, you’re a different kind of partner.

Two, the clipboard system — every week she got a one-page artifact she could forward to her boss without editing. Status, decisions, risks, asks. She never had to translate our work into exec language.

Three, I made her the internal hero. When the B2B vendor portal expansion closed, the story we told inside her company was ‘Consumer BU’s platform choice set up B2B success’ — not ‘Arc4 sold more work.’ She got credit. She opened every door after.

Technical champion: the parent corporate’s director of platform engineering. Different playbook entirely.

One, I gave him the documentation before he asked. Data-flow diagram, tenancy model, deployment audit trail, SIEM-feed spec — all in his format, before the first security review meeting. That signaled I understood his job was to protect the parent, not to bless a vendor.

Two, I let him push back on the architecture. On the schema registry, specifically — he wanted a stricter change-approval process than we’d proposed. I said yes, and we built it his way. His pushback made the system better, and he knew it.

Three, I made his audit cadence predictable. Quarterly security review, same artifacts every time, evidence package pre-assembled. By year two he was using our engagement as the reference model inside the parent for how external platforms should integrate.

The lesson — a commercial champion gives you the next deal; a technical champion makes that deal survive the parent's security review. You need both, and they need different things from you."

e. "What would you do differently?"

"Two things. One, I'd have stood up the schema registry in phase one, not phase three. We paid for schema drift during the Canada/UK clone because we hadn't made the contracts explicit yet. Two, the IDR log was retrofitted after a stakeholder asked 'why did we decide X?' and I couldn't produce a clean answer. It's now a day-one artifact. Both are examples of the same pattern — discipline you institutionalize after you needed it the first time."

f. "Where did this almost fall apart?"

"DNS cutover on the UK launch. A sub-processor changed a header on us mid-deploy and the redirect matrix broke for forty minutes. The deployment checklist caught it — we had a rollback rehearsed and we executed it in twelve minutes. No customer impact beyond a short window of 301 inconsistency. That incident is why the deployment checklist is now non-negotiable on every Arc4 engagement."

g. "How did you measure ROI for the customer?"

"Three horizons. Immediate: organic traffic per locale, conversion on the rebuilt sites, time-to-publish for the ops team. Medium: cost per new-property stand-up — we drove that down 70% from property one to property ten because of the delivery system. Long: revenue attributable to the B2B vendor portal in year two, which was net-new pipeline that the old digital estate couldn't have supported. The parent corporate metric that mattered most was 'new property launch stopped being a board-level risk.'"

h. "Strategically, why did this customer expand with you instead of bringing it in-house?"

"They tried. Parent corporate has a global IT function. Twice they scoped bringing the platform in-house and twice they backed off. Not because we were cheaper — because the delivery system was where the institutional knowledge lived. Schema contracts, IDR log, deployment checklist. Pulling the platform in-house without pulling the delivery system meant starting from zero on governance. The customer figured out that the valuable asset wasn't the code — it was the operational discipline around the code. That's a lesson I carry into how I think about customer value generally."

i. “If we gave you Claude Code for this engagement, where would it have mattered most?”

“Three places. Schema governance — validating that ten property codebases stayed inside the schema contract would have been a Claude Code linter job, not a manual review. SEO pipeline — hreflang and structured data validation were brittle regex in CI; Claude Code would have read the actual schema and the actual intent. Migration moments — every new-locale stand-up was a week of boilerplate; the repeatable parts are exactly what agentic coding does well. Where it would *not* have helped: the stakeholder work. That was never a code problem.”

j. “What does this engagement say about how you’d show up at Anthropic?”

“Scope honestly, even against your own revenue. Institutionalize what worked so the next customer doesn’t pay for the last customer’s hardest week. Treat customer success as a technical problem, not a relationship problem. And understand that the valuable thing you leave behind is usually not the code — it’s the discipline that produced it.”

4. Security & compliance — the parent corporate story

This is an evaluation axis in its own right. Be ready to speak to all of it if prompted; volunteer the one-line version if the panel asks “how did you handle security.”

The one-line version: parent corporate had a Fortune 500 security function that treated every external platform as a risk surface. We gave them evidence before they asked, and we never missed a review cycle.

The seven things they cared about, and what we did:

1. **Data classification at the schema layer, not the UI.** Public product data, PII from inquiries, commercial-sensitive B2B pricing — each entity type carried a classification tag that drove access control, logging, and retention. UI filters were a consequence, not the enforcement.
2. **Identity and access management via the parent IdP.** SSO through the corporate IdP; role-based permissions mapped to entity types, not pages; approval flow for any cross-BU data change.
3. **Tenancy and isolation.** Per-property partitioning with a shared schema. The B2B vendor portal ran on its own auth domain. No cross-BU leakage possible at the query layer — enforced by partition key, not by application code.
4. **Deployment audit trail.** Every deploy traceable to commit, reviewer, stakeholder approval. Rollback rehearsed, not theoretical. Feed into the parent’s SIEM was real-time.
5. **Sub-processor management.** Every dependency (DNS, CDN, image CDN, PIM, DAM) listed with its own DPA on file, its own classification, and a re-review cadence. Parent compliance owned the list; we maintained the technical map behind it.

6. **Regional data residency.** Six countries, three data-residency regimes. The entity graph partition key included locale; no customer data ever crossed a regional boundary without an explicit export event that was logged and reviewable.
7. **Incident discipline.** Runbook per property. DNS cutover near-miss on UK launch was caught by the checklist; 12-minute rollback, no customer impact. Root-cause written up and filed with parent security the same week. Zero data incidents across 18 months and 10 properties.

The move I'm proudest of: we stood up the quarterly security evidence package before parent security asked for one. By Q3 of year one it was the template they were asking other vendors to use.

5. Strategic importance — why this deal mattered (to them and to Arc4)

To the customer:

- **Platform consolidation across three BUs.** Before us, each BU was running its own digital estate on its own stack. The parent had been trying to consolidate for four years and had failed twice. We succeeded not because our tech was better but because the delivery system (schema as contract, IDR log, deployment checklist) gave them the governance discipline an internal mandate couldn't.
- **Board-level risk, de-escalated.** New property launches had been a board-reported risk category because of PR incidents tied to inconsistent content and SEO. By month 18, new-property launch had moved off the board risk register entirely.
- **Net-new revenue via the B2B vendor portal.** The B2B surface wasn't possible on the old stack. First-year revenue through the vendor portal was material enough to be named in the parent's annual report.

To Arc4 (my company):

- **Account expansion from \$140K to \$2.4M — 15x in 18 months, 17 distinct deals, zero churn.** The second-largest engagement in Arc4's history at the time.
- **Methodology that became the company standard.** Every enterprise engagement after this one starts from the delivery system we built here. Proposal win rate on enterprise RFPs jumped after we could show this pattern.
- **Proof that scope honesty compounds.** Rescoping the global merge cost us \$180K in Q1 revenue. It bought us \$2.4M over eighteen months. The internal lesson at Arc4 became: scope hard against your own number, especially early.

The one-sentence strategic frame I'd use on-panel:

"This deal mattered because the valuable artifact we left behind was never the code — it was the operational discipline around the code, and that discipline was what let a Fortune 500 parent consolidate three BUs onto one platform after two prior failed attempts."

6. Numbers you must have memorized

- \$140K → \$2.4M (15x, 18 months)
 - 17 deals, zero churn
 - 10 properties, 6 countries (US, Canada, UK, France, Brazil, + LATAM)
 - 5 stakeholder groups
 - 70% cost reduction per new-property stand-up, property 1 → property 10
 - Zero data incidents, 2 near-misses caught by checklist
 - Phase sequencing: Q4 '23 / Q2 '24 / Q3 '24 / Q1 '25 / Q2 '25
-

7. Anchor phrases to use on-panel

- “Every expansion scoped as its own problem, not as a renewal.”
 - “Scope hard, even if it costs revenue.”
 - “The delivery system became the company standard.”
 - “The valuable asset wasn’t the code — it was the discipline around it.”
 - “We showed up with the documentation parent corporate needed before they asked.”
-

8. Kill list (don’t say)

- “It was really complicated” (describe the complication)
 - “We just” (never — it always undersells)
 - “Obviously” (nothing is obvious to them)
 - Jargon without a plain-English gloss: hreflang, ISR, SIEM all need a half-sentence definition if the panel looks blank
 - Name-dropping Yext more than twice — the lesson is the pattern, not the vendor
-

9. Technical depth territories (where to dig if they push beyond CMS + API)

The recruiter feedback called this out explicitly as evaluation criteria. Pick one or two to go deep on — don’t lecture across all five.

1. **Entity modeling & taxonomy design.** Why typed nodes over flat catalog. Relationship cardinality decisions (a Product has many SKUs, belongs to one Category, has many Articles, is localized to many Locales). Where we got it wrong the first time (we initially made Locale a field, not an entity — had to refactor in month six).
2. **Relevance engineering without vectors.** Per-locale eval sets. P@1 and MRR regression testing. Synonym and definition review cadence owned by marketing ops. Why we held off on embeddings and what would trigger a revisit.
3. **Identity resolution as first-class infrastructure.** Deterministic rules on part-number + region + variant hash, fuzzy fallback with a human review queue.

What an IDR break looked like in practice (Canada clone schema drift caught four duplicate Product entities before launch).

4. **Schema evolution discipline.** Additive-by-default. Breaking changes require a migration script, rehearsed in staging, forward-compatible for one release before removal. Every schema PR reviewed by both Arc4 and parent corporate's platform eng.
 5. **Cross-surface consistency from one graph.** Same entity graph feeds on-site search, hreflang-aware SEO, JSON-LD structured data, and an LLM-ready JSON surface. No surface is allowed to maintain a side-schema, ever. That one rule is what made the 2024 LLM-payload project three weeks instead of six months.
-

10. The “I don’t know” discipline

The recruiter explicitly said: **say you don’t know if you don’t know.** Five-beat template when it happens:

1. **Name it.** “I don’t know the answer to that.”
2. **Say what you’d need.** “To answer it properly I’d want to pull the actual eval numbers from our Q3 relevance report.”
3. **Commit a timeline.** “I can get you that within the day.”
4. **Give the adjacent known thing.** “What I can tell you right now is how we designed the eval set and what the top failure modes were.”
5. **Check in.** “Does that work, or is the specific number the important thing?”

Ready-to-deploy lines:

“Honestly, I don’t remember the exact P@1 number for the French locale — I’d be guessing. I can pull it after the call. What I do remember is the failure mode we saw most often was compound-noun queries in French breaking our synonym matcher, and that’s what drove the relevance review cadence we ended up shipping.”

“I didn’t personally own that piece — our data engineer did. If you want the real answer I’d want to bring her into a follow-up. I can describe the shape of the decision but I don’t want to make up the specifics.”

“That’s a sharper question than I have a clean answer for. Let me think out loud for a second rather than give you a rehearsed answer.”

11. T-minus checklist

T-24h: Re-read S5, S6, S7 narratives out loud. Time yourself on the 10-minute walk. Write down the three most likely probes and rehearse answers.

T-2h: Deck pulled up. Coffee. No new additions.

T-0: Open with slide 1. Land the \$140K → \$2.4M number within the first minute. Let them interrupt — the story holds if they pick any slide to go deep on.